Laboration 1

Markus Svan

2021/11/23

Contents

1 Introduction **3**

2 Algorithm Explanation **3**
- 2.1 Bubble Sort . . . 3
- 2.2 Insertion Sort . . . 3
- 2.3 Quick Sort . . . 4
- 2.4 Linear Search . . . 4
- 2.5 Binary Search . . . 5

3 Code Justification **5**

4 Algorithm Analysis **6**
- 4.1 Bubble Sort best case . . . 6
- 4.2 Bubble Sort worst case . . . 6
- 4.3 Bubble Sort average case . . . 6
- 4.4 Insertion Sort best case . . . 7
- 4.5 Insertion Sort worst case . . . 7
- 4.6 Insertion Sort average case . . . 7
- 4.7 Quick Sort best case . . . 8
- 4.8 Quick Sort worst case . . . 8
- 4.9 Quick Sort average case . . . 8
- 4.10 Linear Search best case . . . 9
- 4.11 Linear Search worst case . . . 9
- 4.12 Linear Search average case . . . 9
- 4.13 Binary Search best case . . . 10
- 4.14 Binary Search worst case . . . 10
- 4.15 Binary Search average case . . . 10

5 Problems **11**

6 Conclusion **11**

1 Introduction

This report is about testing different sorting and search algorithms. The different sorting algorithms that are being tested in this report are the following `Bubble Sort`, `Insertion Sort`, and `QuickSort` The search algorithms that get tested are `Linear Search` and `Binary Search`
The report will look at the complexity of each algorithm in "big O" notation which is used to explain the expected time complexity of an algorithm based on the size of the list needing to be sorted which is noted down as `n`

2 Algorithm Explanation

2.1 Bubble Sort

Bubble Sort steps through the given list of items from the lowest index to the next highest index and compares the element it is currently looking at with the element one index higher. If the element at the current index is bigger than the index adjacent to it they swap places and the algorithm continues. After each iteration of the list it goes back to the lowest index but will then go up to the past highest index minus one. This continues until either there's an iteration where no swaps had to be made, meaning the list is already sorted or until the lowest index and past highest index become the same.
The best case for Bubble Sort is if the list is already sorted. This results in a complexity of `O(n)` as the algorithm will step through the list once and then return since it didn't make a single swap.
The worst case for Bubble Sort is if the list is already sorted in the opposite direction. This results in a complexity of $O(n^2)$ because the algorithm has to step through the list making a swap on every single iteration of the loop
The average case for Bubble Sort is when the algorithm is given a random list. After benchmarking Bubble Sort average case through `analyze.c > bench_bubble` the measured times somehow result in a time complexity worse than $O(n^2)$.

2.2 Insertion Sort

Insertion Sort steps through the given list of items from the next lowest index to the highest index. On each step it stores the value of the current index. If the stored value is lower than the value one index before it'll enter a loop that goes from the current index towards the start of the list, constantly overwriting the new current index with the value before it until the stored value is no longer smaller than the new index and finally put the stored value on the index of the second loop.
The best case for Insertion Sort is an already sorted list. This results in a complexity of `O(n)` because the algorithm will step through the list once and never have to move any values before it finishes.
The worst case for Insertion Sort is an array sorted in the opposite direction. This results in a complexity of $O(n^2)$ due to the algorithm looping through the

entire list of items and has to constantly move values during the second loop.
The average case for Insertion Sort is a random list. After benchmarking Insertion Sort average case through `analyze.c > bench_insertion` the measured times while faster than worst case still closely resemble a complexity of $O(n^2)$.

2.3 Quick Sort

Quick sort selects the last element in the list as a pivot element and puts all values smaller than the pivot element to the left of it in the list and all the values bigger than the pivot element to the right of it in the list. The pivot element is placed correctly in the list by comparing it to every element in the list (including itself) and counting the amount of times the pivot element is bigger or equal. The amount of times the pivot was bigger or equal is also the pivot elements index in the list! After having placed the pivot element on the correct the algorithm then continues on the new smaller lists to the left and or right of the pivot element. The best case for Insertion Sort is when the pivot element is the middle element in each list section. Sadly implementing a function to make the best case list was very hard. However I can give an example list that would be best case of 9 values. `1 3 4 6 8 9 2 7 5` this list is a best case list as 5 is the middle element and will be picked as a pivot. And in the next recursive call of the function 2 and 7 will be the pivot element for the new respective sections.
The worst case for Quick Sort is an already sorted list. The reason for this is that the pivot element will already be in the last and correct position. So there would be no splitting of the list into smaller sections. This results in a complexity of $O(N^2)$. The average case for Quick Sort is a random list. After benchmarking quick sort average case through `analyze.c > bench_quick` it conforms to a complexity of `O(nLog(n))`.
The theoretical best case for Quick Sort is `O(nLog(n))`.

2.4 Linear Search

Linear search compares every item in the list from the lowest index to either the highest index or when the queried value is found. The best case for Linear Search is when the value being searched for is the first item in the list. This results in a copmplexity of `O(1)` as there is just one comparison before the value being found it is constant time in best case.
The worst case for Linear Search is when the value is not in the list. This results in a complexity of `O(n)` as it steps through every single item in the list before not finding the element.
The average case for Linear Search is when the value is somewhere in the list, preferably not at the start or end but closer to the middle. The complexity for Linear Search average case is less than `O(n)` but higher than `O(1)` and could be represented by roughly `O(n/2)`.

2.5 Binary Search

Binary Search is the only algorithm out of the ones listed which has a prerequisite, which is that the list has to be sorted. Binary search checks the middle element of the list and compares it to the queried value, if the search query is bigger than the middle element it will "raise the floor" to the middle elements index + 1. Efectively cutting the list in half, and will then compare the new middle element in the "new list" to the search query and repeat until the right value is found or that there's no more dividing the list in half. Of course if the search query is smaller the "Roof" will be lowered instead to the middle elements index - 1.
The best case for Binary Search is when the first calculated middle element is the same as the search query. This results in a complexity of `O(1)` since binary search will finish instantly on the first comparison.
The worst case for Binary Search is when the search query is the last element in the list. This results in a complexity of `O(log(n))` because of the continuous division of 2 on the list.
The average case for Binary Search is when the search query is a random element potentially in a random list. This results in a complexity in between best case and worst case.

3 Code Justification

The code uses the same benchmark function called `bench_press` for each algorithm. It recieves a function pointer to the algorithm it has to run and based on the parameters which are meant to be fed into the algorithm decides if it's a sorting or search algorithm. The function then measures the time of each function `ITERATION` amount of times which is decided in `analyze.h` and stores the average in the results parameter sent in through the function call. Each algorithm has their own function to generate the best, worst and average case lists. The time measurement is started and stopped right before and after the algorithm call to measure no other instructions.

4 Algorithm Analysis

4.1 Bubble Sort best case

[!ht] All the following tables are average results from 10 iterations.
Bubble Sort best case seems to conform to `O(n)`. This makes sense as the algorithm steps through the list once (compares n elements) before declaring it sorted.

```
*****************************************************************************************
                                bubble sort: best
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    8.979000e-07  9.976667e-08  1.948568e-10  3.507422e-09  1.753711e-09  3.425217e-12  6.689876e-15
1024   1.747000e-06  1.747000e-07  1.706055e-10  3.412109e-09  1.706055e-09  1.666069e-12  1.627021e-15
2048   3.477100e-06  3.161000e-07  1.543457e-10  3.395605e-09  1.697803e-09  8.290052e-13  inf
4096   7.379600e-06  6.149667e-07  1.501383e-10  3.603320e-09  1.801660e-09  4.398584e-13  inf
8192   1.389290e-05  1.068685e-06  1.304547e-10  3.391821e-09  1.695911e-09  2.070203e-13  inf
16384  2.769990e-05  1.978564e-06  1.207620e-10  3.381335e-09  1.690668e-09  1.031902e-13  inf
```

Figure 1: Bubble Sort best case benchmar results

4.2 Bubble Sort worst case

[!ht] Bubble Sort worst case seems to conform to `O(n²)`. This makes sense because the algorithm has to swap every single element in the list before it's sorted.

```
*****************************************************************************************
                                bubble sort: worst
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    4.133091e-04  4.592323e-05  8.969382e-08  1.614489e-06  8.072443e-07  1.576649e-09  3.079393e-12
1024   1.687755e-03  1.687755e-04  1.648198e-07  3.296396e-06  1.648198e-06  1.609569e-09  1.571844e-12
2048   6.711813e-03  6.101648e-04  2.979320e-07  6.554504e-06  3.277252e-06  1.600221e-09  inf
4096   2.650916e-02  2.209097e-03  5.393303e-07  1.294393e-05  6.471963e-06  1.580069e-09  inf
8192   1.065857e-01  8.198900e-03  1.000842e-06  2.602190e-05  1.301095e-05  1.588251e-09  inf
16384  4.330325e-01  3.093089e-02  1.887872e-06  5.286041e-05  2.643021e-05  1.613172e-09  inf
```

Figure 2: Bubble Sort worst case benchmark results

4.3 Bubble Sort average case

[!ht] Bubble sort average case seems to conform most closely to `O(n²)`. This does make some sense as it should be comparing and swapping elements roughly 50% of the time, meaning it's more operations than best case but should be less operations than worst case. However the shocking result is that average case took longer to compute than worst case. While I am unable to give an objective answer without speculation as for why this result is the way it is I will try to explain it in section 5 Probelms.

```
*****************************************************************************************************
                                        bubble sort: average
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    4.084079e-04  4.537866e-05  8.863019e-08  1.595343e-06  7.976717e-07  1.557952e-09  3.042876e-12
1024   1.554585e-03  1.554585e-04  1.518150e-07  3.036300e-06  1.518150e-06  1.482568e-09  1.447820e-12
2048   5.995659e-03  5.450599e-04  2.661425e-07  5.855136e-06  2.927568e-06  1.429477e-09  inf
4096   2.662011e-02  2.218342e-03  5.415875e-07  1.299810e-05  6.499050e-06  1.586682e-09  inf
8192   1.316938e-01  1.013029e-02  1.236608e-06  3.215180e-05  1.607590e-05  1.962390e-09  inf
16384  5.776241e-01  4.125886e-02  2.518241e-06  7.051075e-05  3.525538e-05  2.151817e-09  inf
```

Figure 3: Bubble Sort average case benchmark results

4.4 Insertion Sort best case

[!ht] Insertion Sort best case seems to conform to `O(n)`. This makes sense as the algorithm steps through the lise once before declaring the list as sorted.

```
*****************************************************************************************************
                                        insertion sort: best
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    1.726800e-06  1.918667e-07  3.747396e-10  6.745313e-09  3.372656e-09  6.587219e-12  1.286566e-14
1024   2.747200e-06  2.747200e-07  2.682812e-10  5.365625e-09  2.682812e-09  2.619934e-12  2.558529e-15
2048   5.466700e-06  4.969727e-07  2.426625e-10  5.338574e-09  2.669287e-09  1.303363e-12  inf
4096   1.089780e-05  9.081500e-07  2.217163e-10  5.321191e-09  2.660596e-09  6.495595e-13  inf
8192   2.258200e-05  1.737077e-06  2.120455e-10  5.513184e-09  2.756592e-09  3.364980e-13  inf
16384  4.426720e-05  3.161943e-06  1.929897e-10  5.403711e-09  2.701855e-09  1.649082e-13  inf
```

Figure 4: Insertion Sort best case benchmark results

4.5 Insertion Sort worst case

[!ht] Insertion Sort worst case seems to conform to `O(n²)`. This makes sense as the algorithm has to move and compare the highest possible amount of times, which due to the nested for loops is n^2.

```
*****************************************************************************************************
                                        insertion sort: worst
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    2.966003e-04  3.295559e-05  6.436638e-08  1.158595e-06  5.792975e-07  1.131440e-09  2.209844e-12
1024   1.064972e-03  1.064972e-04  1.040012e-07  2.080024e-06  1.040012e-06  1.015637e-09  9.918327e-13
2048   4.333575e-03  3.939614e-04  1.923639e-07  4.232007e-06  2.116003e-06  1.033205e-09  inf
4096   1.701341e-02  1.417784e-03  3.461388e-07  8.307331e-06  4.153665e-06  1.014078e-09  inf
8192   6.822045e-02  5.247727e-03  6.405917e-07  1.665538e-05  8.327692e-06  1.016564e-09  inf
16384  2.743118e-01  1.959370e-02  1.195905e-06  3.348533e-05  1.674266e-05  1.021891e-09  inf
```

Figure 5: Insertion Sort worst case benchmark results

4.6 Insertion Sort average case

[!ht] Insertion Sort average case seems to conform the most to `O(n²)` but on a downwards trend. This makes sense as the algorithm compares a randomly generated list of elements, so if truly random it should on average insert elements somewhere in the middle of the sorted elements before it. While these times aren't slower than their worst case counterpart they still seem very close to the worst case times. I will try to explain this in section 5 Problems.

```
*****************************************************************************************************
                                        insertion sort: average
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    1.434410e-04  1.593789e-05  3.112869e-08  5.603164e-07  2.801582e-07  5.471840e-10  1.068719e-12
1024   5.401823e-04  5.401823e-05  5.275218e-08  1.055044e-06  5.275218e-07  5.151580e-10  5.030840e-13
2048   2.166858e-03  1.969871e-04  9.618510e-08  2.116072e-06  1.058036e-06  5.166192e-10  inf
4096   8.516357e-03  7.096964e-04  1.732657e-07  4.158377e-06  2.079189e-06  5.076144e-10  inf
8192   3.430482e-02  2.638832e-03  3.221231e-07  8.375201e-06  4.187600e-06  5.111817e-10  inf
16384  1.373274e-01  9.809101e-03  5.987000e-07  1.676360e-05  8.381800e-06  5.115845e-10  inf
```

Figure 6: Insertion Sort average case benchmark results

4.7 Quick Sort best case

[!ht] I was unable to generate a best case array for Quick Sort so there are no results to show for it.

4.8 Quick Sort worst case

[!ht] Quick Sort worst case conforms to `O(n²)`. This makes sense because on every failed recursive splitting of the list no improvement was made from trying to divide the list.

```
*****************************************************************************************************
                                          quick sort: worst
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    3.710052e-04  4.122280e-05  8.051328e-08  1.449239e-06  7.246195e-07  1.415273e-09  2.764204e-12
1024   1.528997e-03  1.528997e-04  1.493161e-07  2.986323e-06  1.493161e-06  1.458165e-09  1.423990e-12
2048   6.007906e-03  5.461732e-04  2.666862e-07  5.867095e-06  2.933548e-06  1.432396e-09  inf
4096   2.370630e-02  1.975525e-03  4.823059e-07  1.157534e-05  5.787671e-06  1.413006e-09  inf
8192   9.464680e-02  7.280523e-03  8.887357e-07  2.310713e-05  1.155356e-05  1.410347e-09  inf
16384  3.790758e-01  2.707684e-02  1.652639e-06  4.627390e-05  2.313695e-05  1.412167e-09  inf
```

Figure 7: Quick Sort worst case benchmark results

4.9 Quick Sort average case

[!ht] Quick Sort average case seems to conform to either `O(nlogn)` or `O(n/2)`. This result makes sense because the algorithm will sometimes pick the best, worst or average pivot for it's list.

```
*****************************************************************************************************
                                          quick sort: average
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    3.420410e-05  3.800456e-06  7.422765e-09  1.336098e-07  6.680488e-08  1.304783e-10  2.548404e-13
1024   7.242400e-05  7.242400e-06  7.072656e-09  1.414531e-07  7.072656e-08  6.906891e-11  6.745011e-14
2048   1.465507e-04  1.332279e-05  6.505269e-09  1.431159e-07  7.155796e-08  3.494041e-11  inf
4096   3.231838e-04  2.693198e-05  6.575191e-09  1.578046e-07  7.890229e-08  1.926326e-11  inf
8192   6.912008e-04  5.316929e-05  6.490392e-09  1.687502e-07  8.437510e-08  1.029969e-11  inf
16384  1.524678e-03  1.089056e-04  6.647070e-09  1.861180e-07  9.305898e-08  5.679869e-12  inf
```

Figure 8: Quick Sort average case benchmark results

4.10 Linear Search best case

[!ht] Linear Search best case conforms to `O(1)`. This constant time makes sense since the best case is immediately finding the search query and returning true.

```
*****************************************************************************************************
                                        linear search: best
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    2.630000e-08  2.922222e-09  5.707465e-12  1.027344e-10  5.136719e-11  1.003265e-13  1.959503e-16
1024   1.650000e-08  1.650000e-09  1.611328e-12  3.222656e-11  1.611328e-11  1.573563e-14  1.536682e-17
2048   1.610000e-08  1.463636e-09  7.146662e-13  1.572266e-11  7.861328e-12  3.838539e-15  inf
4096   1.650000e-08  1.375000e-09  3.356934e-13  8.056641e-12  4.028320e-12  9.834766e-16  inf
8192   1.820000e-08  1.400000e-09  1.708984e-13  4.443359e-12  2.221680e-12  2.712011e-16  inf
16384  1.790000e-08  1.278571e-09  7.803781e-14  2.185059e-12  1.092529e-12  6.668270e-17  inf
```

Figure 9: Linear Search best case benchmark results

4.11 Linear Search worst case

[!ht] Linear Search worst case seems to conform the best to `O(n)`. This makes sense due to the value either not being in the list but the search algorithm still having to go through the entire list to be certain the value isn't in the list.

```
*****************************************************************************************************
                                        linear search: worst
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    9.591000e-07  1.065667e-07  2.081380e-10  3.746484e-09  1.873242e-09  3.658676e-12  7.145852e-15
1024   1.871200e-06  1.871200e-07  1.827344e-10  3.654688e-09  1.827344e-09  1.784515e-12  1.742691e-15
2048   3.719900e-06  3.381727e-07  1.651234e-10  3.632715e-09  1.816357e-09  8.868933e-13  inf
4096   7.408300e-06  6.173583e-07  1.507222e-10  3.617334e-09  1.808667e-09  4.415691e-13  inf
8192   1.482740e-05  1.140569e-06  1.392296e-10  3.619971e-09  1.809985e-09  2.209455e-13  inf
16384  2.968350e-05  2.120250e-06  1.294098e-10  3.623474e-09  1.811737e-09  1.105797e-13  inf
```

Figure 10: Linear Search worst case benchmark results

4.12 Linear Search average case

[!ht] Linear Search average case conforms the most to `O(n/2)`. This is due to the search query value existing at a random location in the list, and the average of all the possible locations becomes the center of the list. Meaning it makes sense that the complexity is for half the list.

```
*****************************************************************************************************
                                        linear search: average
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    2.305800e-06  2.562000e-07  5.003906e-10  9.007031e-09  4.503516e-09  8.795929e-12  1.717955e-14
1024   1.104100e-06  1.104100e-07  1.078223e-10  2.156445e-09  1.078223e-09  1.052952e-12  1.028273e-15
2048   1.951000e-07  1.773636e-08  8.660334e-12  1.905273e-10  9.526367e-11  4.651546e-14  inf
4096   2.223500e-06  1.852917e-07  4.523722e-11  1.085693e-09  5.428467e-10  1.325309e-13  inf
8192   4.959800e-06  3.815231e-07  4.657264e-11  1.210889e-09  6.054443e-10  7.390678e-14  inf
16384  1.247130e-05  8.908071e-07  5.437055e-11  1.522375e-09  7.611877e-10  4.645921e-14  inf
```

Figure 11: Linear Search average case benchmark results

4.13 Binary Search best case

[!ht] Binary Search best case conforms to `O(1)`. This constant time is due to the first element compared is the correct value every time so the size of the list isn't relevant.

```
*****************************************************************************************************
                                        binary search: best
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    2.970000e-08  3.300000e-09  6.445312e-12  1.160156e-10  5.800781e-11  1.132965e-13  2.212822e-16
1024   2.040000e-08  2.040000e-09  1.992187e-12  3.984375e-11  1.992188e-11  1.945496e-14  1.899898e-17
2048   1.920000e-08  1.745455e-09  8.522727e-13  1.875000e-11  9.375000e-12  4.577637e-15  inf
4096   2.230000e-08  1.858333e-09  4.536947e-13  1.088867e-11  5.444336e-12  1.329184e-15  inf
8192   2.120000e-08  1.630769e-09  1.990685e-13  5.175781e-12  2.587891e-12  3.159046e-16  inf
16384  2.020000e-08  1.442857e-09  8.806501e-14  2.465820e-12  1.232910e-12  7.525086e-17  inf
```

Figure 12: Binary Search best case benchmark results

4.14 Binary Search worst case

[!ht] Binary Search worst case conforms to `O(logn)`. This very quick worst case is because the correct value is placed as the maximum index of the list. Binary Search similarly to Quick Sort divides the list in halves, but unlike Quick Sort Binary Search succeeds in the division every time until the element is found or declared not in the list.

```
*****************************************************************************************************
                                        binary search: worst
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    6.160000e-08  6.844444e-09  1.336806e-11  2.406250e-10  1.203125e-10  2.349854e-13  4.589558e-16
1024   5.740000e-08  5.740000e-09  5.605469e-12  1.121094e-10  5.605469e-11  5.474091e-14  5.345792e-17
2048   5.950000e-08  5.409091e-09  2.641158e-12  5.810547e-11  2.905273e-11  1.418591e-14  inf
4096   6.210000e-08  5.175000e-09  1.263428e-12  3.032227e-11  1.516113e-11  3.701448e-15  inf
8192   6.710000e-08  5.161538e-09  6.300706e-13  1.638184e-11  8.190918e-12  9.998679e-16  inf
16384  7.230000e-08  5.164286e-09  3.152030e-13  8.825684e-12  4.412842e-12  2.693385e-16  inf
```

Figure 13: Binary Search worst case benchmark results

4.15 Binary Search average case

[!ht] Binary Search average case conforms the most to `O(logn)`. This makes sense because the amount of divisions needed to get to a lot of values in the list is the same as the worst case. The reason for this is that the divisions let the algorithm step through the list as a binary tree. The values at the bottom are the furthest away but there are more of them than values found right away.

```
*****************************************************************************************************
                                        binary search: average
~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
size   time T(s)     T/logn        T/nlogn       T/(n/2)       T/n           T/n^2         T/n^3
512    5.830000e-08  6.477778e-09  1.265191e-11  2.277344e-10  1.138672e-10  2.223969e-13  4.343688e-16
1024   6.100000e-08  6.100000e-09  5.957031e-12  1.191406e-10  5.957031e-11  5.817413e-14  5.681068e-17
2048   6.300000e-08  5.727273e-09  2.796520e-12  6.152344e-11  3.076172e-11  1.502037e-14  inf
4096   7.110000e-08  5.925000e-09  1.446533e-12  3.471680e-11  1.735840e-11  4.237890e-15  inf
8192   8.260000e-08  6.353846e-09  7.756160e-13  2.016602e-11  1.008301e-11  1.230836e-15  inf
16384  7.900000e-08  5.642857e-09  3.444127e-13  9.643555e-12  4.821777e-12  2.942979e-16  inf
```

Figure 14: Binary Search average case benchmark results

5 Problems

[!ht] Why is the average case for bubble sort on some occasions slower than the theoretical worst case? The speculated answer I've arrived at is that the branch prediction on processors learns to predict the guaranteed swapping of both bubble and insertion sort. Every time the processor predicts an if statement (including conditions for loops) it results in a high performance increase compared to when it guesses incorrectly. On average cases the processor can't predict if it should or shouldn't swap due to the list being random. On each failed prediction the processor has to rebuild the pipeline resulting in a huge slowdown of performance.
Quick Sort best case is a "Binary Search Tree in preorder". I was sadly unable to make a function able to generate lists sorted this way in time for this assignment

6 Conclusion

During this assignment I learned the meaning and usage of "Big O notation". I also learned how to code different sorting and search algorithms. I learned how to think recursively with Quick Sort. The worst case for sorting any list often conforms to n^2 unless you're using a `O(n+r)` algorithm like Counting Sort. Binary Search is able to be so quick due to the list already being sorted letting the algorithm draw conclusions not possible on unsorted lists for which you'd have to use Linear Search.